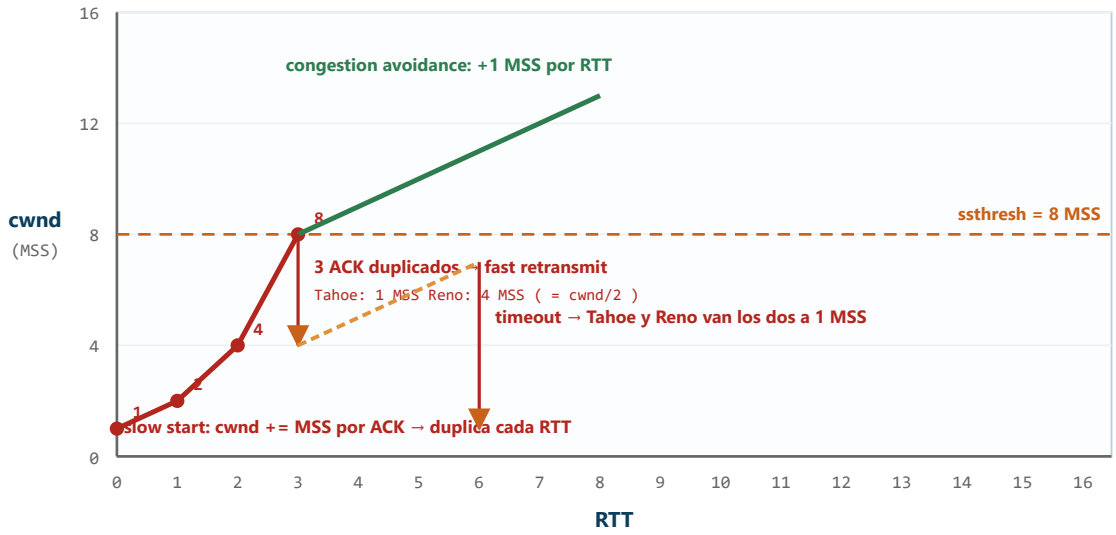
Clase 20 — Control de flujo vs. control de congestión

Bloque: C (Transporte) · **Semana:** 7 · **Duración estimada:** 100 min **Correlativa:** Clase 19 (cómo garantiza TCP la fiabilidad) · **Sigue con:** Clase 21 (control de congestión en TCP: slow start, AIMD y ECN) **Material de referencia:** Tanenbaum cap. 6 (§6.7) · RFC 9293, RFC 7323, RFC 6928, RFC 5681, RFC 9438

Evolucion de la ventana de congestion



Tahoe vuelve a 1 MSS ante cualquier perdida.

Reno solo cae a cwnd/2 con 3 ACK duplicados; ante un timeout cae igual que Tahoe.

🎯 Al terminar esta clase vas a poder

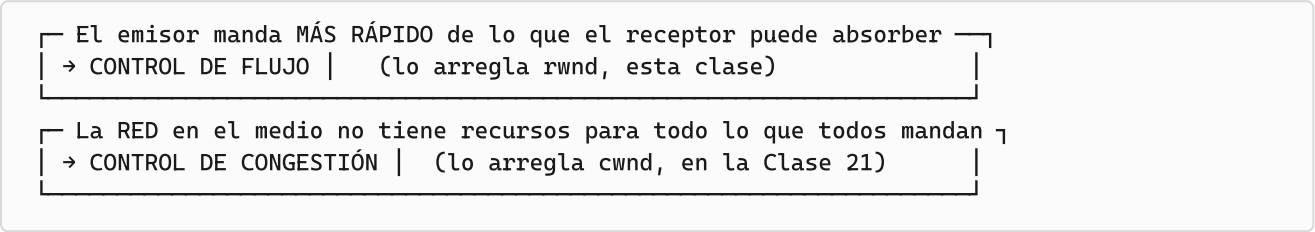
1. Distinguir **control de flujo** de **control de congestión** y decir quién controla cada uno y con qué variable.
2. Explicar cómo viaja el `rwnd` en la cabecera TCP y por qué el emisor **no puede mandar más que el** `rwnd`.
3. Justificar por qué la ventana efectiva es el **mínimo** de `cwnd` y `rwnd` y no la suma.
4. Describir qué pasa cuando el `rwnd` llega a **cero**, qué son los *zero-window probes* y por qué eso **no** es control de congestión.
5. Explicar el problema de los **buffers de socket** en los dos extremos y por qué el del emisor **también** es un problema de flujo.
6. Calcular el **throughput** de una conexión cuando el límite es el `rwnd`, y decir cuál de los dos límites manda.

> **Prerequisito del examen:** `2009-07 P1` (10 pts — flujo vs. congestión, y cómo detecta TCP la congestión), `2010-02 P5` (7 pts — el mecanismo de control de flujo de TCP), `2011-03 P4` (10 pts — el diagrama con `cwnd`), `2010-12 P4b` (los timers, que son del otro lado de la misma ecuación).

PARTE 1 — TEORÍA

1.1 Los dos problemas

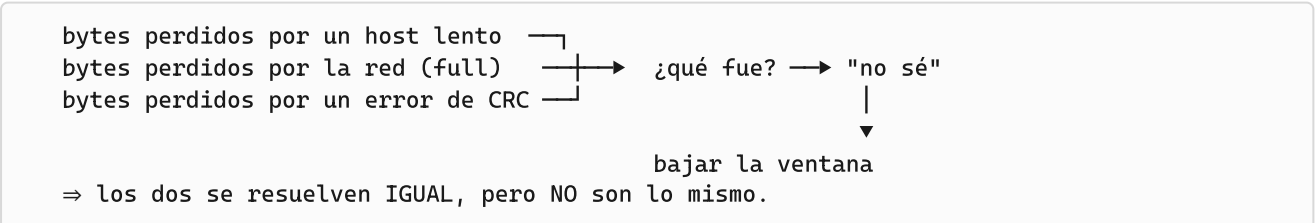
Imaginá una descarga. Tu disco de 500 Mbps no te sirve de nada si el enlace de tu casa da 10 Mbps. Y al revés: un enlace de 100 Mbps que compartís con 40 personas te va a parecer lento. Son **dos problemas distintos**:



La tabla que usa el profesor para diferenciarlos. Esta es, literal, la que hay que aprender de memoria porque es la respuesta de `2009-07 P1a`:

	Control de flujo	Control de congestión
El problema es de	un **host lento**	**toda la red**
El síntoma es	el buffer del receptor se llena	los buffers de **los routers** se llenan
La pérdida significa	"no leíste rápido"	"**la red está saturada**"
La ventana la controla	el **receptor** (`rwnd`)	el **emisor**, estimando (`cwnd`)
Se arregla con	ACK diferidos / zero-window probe	**reducir el ritmo de envío**
Le importa el BDP	sí, mucho	sí, pero como techo físico

La evidencia de que confundirlos es el error típico del examen. La razón de fondo es esta: **TCP no puede distinguir una pérdida por buffer lleno del receptor de una pérdida en la red.** Para TCP, un paquete perdido es un paquete perdido. Por eso el mismo mecanismo (disminuir la ventana) sirve para las dos cosas, y por eso la interpretación que TCP hace es conservadora:



Y esto conecta con la pregunta 1c de `2009-07`, la más fina del tema: **¿la pérdida siempre implica congestión? No.** También por una capa de enlace defectuosa, un buffer del receptor lleno o un error de CRC. TCP **asume** que sí, porque asumir lo contrario sería peor en promedio: castigar a la red de vez en cuando sale más barato que saturarla.

1.2 El `rwnd`: el campo *window* de la cabecera

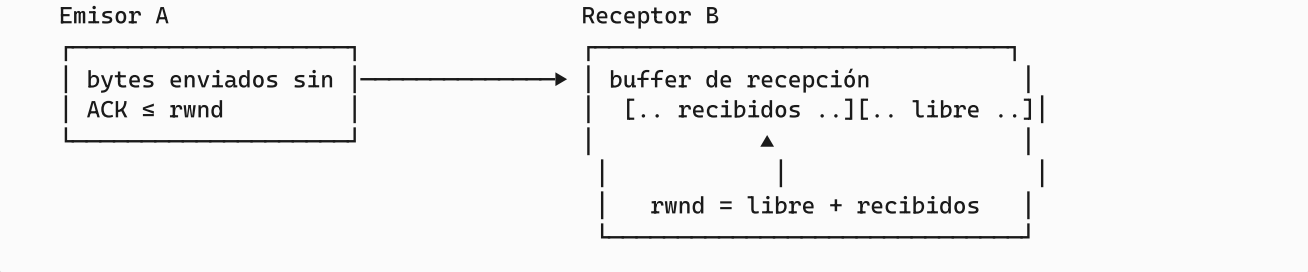
`rwnd` (*receive window*) es la cantidad de bytes que el receptor dice que le entran en su buffer **a partir del byte que está esperando**. Viaja en el campo `window` de **cada** segmento TCP, y es de **16 bits** (0 a 65 535), multiplicado por el factor de escalado si se negoció *Window Scale*.

La cuenta del número de ventana. El receptor no anuncia "me quedan 40000 bytes de buffer". Anuncia **cuántos bytes quiere recibir a partir del próximo que espera**:

```
rwnd = (espacio libre en el buffer) + (bytes ya recibidos sin entregar a la aplicación)
```

O sea, el `rwnd` cubre **los dos** casos: lo que ya llegó esperando `read()` y lo que puede llegar todavía. Esto es lo que hace que sea una cota *superior* y no una medida instantánea.

La restricción dura: el emisor **no puede tener más de `rwnd` bytes sin confirmar**.



1.3 La ventana efectiva: por qué el mínimo y no la suma

```
ventana efectiva = min(cwnd, rwnd)
                   ─────    ─────
                congestión   flujo
```

¿Por qué el mínimo? Porque **las dos son cotas superiores independientes**:

- el emisor no puede mandar más de lo que **la red aguanta** (`cwnd`), y
- no puede mandar más de lo que **el receptor aguanta** (`rwnd`).

¿Por qué no la suma? Este es el punto fino que más se pregunta. Si la ventana efectiva fuera `cwnd + rwnd`, el excedente que la red no pudo tomar **se acumularía en el buffer del receptor y lo desbordaría**. Y lo desbordaría **justo cuando la red va peor**, que es exactamente cuando el buffer del receptor ya está más lleno. El `min` es el que garantiza que **nadie se desborda**: ni la red (por `cwnd`), ni el receptor (por `rwnd`).

```
Ventana efectiva = cwnd + rwnd (¡MAL!)

A ──[──────────── cwnd ────────────][──── rwnd ────]──▶ B
                                                  / buffer de B
                                                 / se llena con lo que
                                                / la red NO pudo tomar
                                                ↓
                                          OVERFLOW de B

Ventana efectiva = min(cwnd, rwnd) (✓ correcto)

A ──[──── min(cwnd, rwnd) ────]──▶ B
     todo lo que sale entra, y entra todo lo que sale
```

En los exámenes los dos los llaman "ventana efectiva" o "ventana de congestión" según el caso. Si el enunciado habla de **una sola** ventana, asumí que `rwnd` es grande y el límite es `cwnd`.

> Todo el desarrollo del `cwnd` — slow start, congestion avoidance, fast recovery, ECN — está en la **Clase 21**. Esta clase se limita a explicar **qué es** y **por qué son dos cosas**.

1.4 Zero window: el receptor que no lee

El caso límite del control de flujo: el receptor **dejó de leer** y su buffer está lleno.

```
t0  B anuncia rwnd = 20 000      A manda 20 000 bytes      buffer de B: LLENO
t1  A manda otros 20 000         B NO asienta              rwnd = 0
t2  A no puede mandar nada                          ventana efectiva = min(cwnd, 0) = 0
t3  A queda en silencio...
```

Si A se quedara quieto para siempre, la conexión se **cuelga**, aunque el otro lado esté perfectamente sano. Para eso existe el **persistence timer** (`RFC 9293` §8.6): A manda **zero-window probes** (*sondas de ventana cero*).

```
A ─────[1 byte, seq=next]──▶ B    ¿podés volver a leer?
◀─────[ACK, rwnd=0]─────── A    "todavía no" → esperar más
    (espera el doble: 1 s, 2 s, 4 s, 8 s... hasta un tope)
A ─────[1 byte, seq=next]──▶ B
◀─────[ACK, rwnd=16384]─── A    "ya, dale" → rwnd ≠ 0 → reanuda
```

Las características de las sondas, que hay que saber:

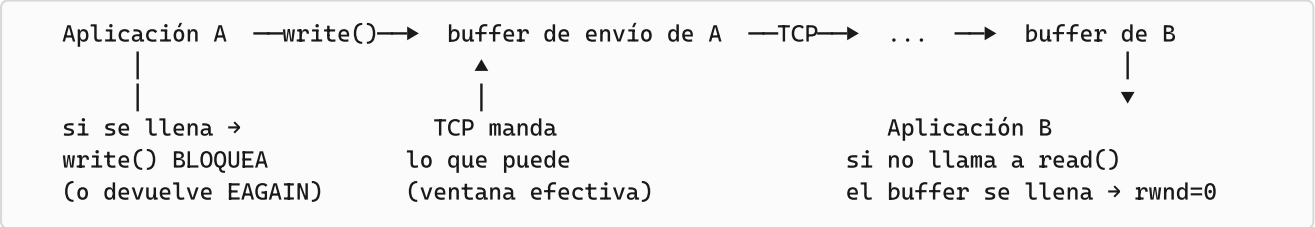
Aspecto	Detalle
Tamaño	**1 byte** (o el último byte recibido)
Cuándo	cuando expira el *persist timer*, no antes
Periodicidad	**exponencial**: RTO, 2·RTO, 4·RTO… hasta un tope (~60 s)
Para qué	no es un retransmit: es una **consulta**
Si B responde con `rwnd = 0`	A vuelve a esperar otro intervalo
Riesgo	el **deadlock permanente**: si B cerró el socket, A queda sondeando para siempre (el *keepalive* es lo que corta esto)

Y acá está lo importante para el examen:

> ⚠️ **Esto NO es control de congestión.** Es control de flujo, y no confundirlo es explícitamente lo que se busca en `2009-07 P1a`. **La red está perfecta**: nadie está saturado, no hay congestión de ningún tipo. Lo que está lento es la **aplicación del otro lado**, que no está llamando a `read()`. Si TCP bajara el `cwnd` por esto, estaría castigando a la red por un problema del receptor, y además **no arreglaría nada**: el `cwnd` no puede hacer que una aplicación lea más rápido. **Por eso son mecanismos separados.**

1.5 El otro lado: los buffers de socket del emisor

El `rwnd` protege el buffer **del receptor**. Pero hay un tercer buffer en la ecuación, y es el que se olvida: el **buffer de envío del emisor**.



El caso del emisor. Si A escribe más rápido de lo que la conexión puede sacar (porque el `rwnd` de B es chico, o porque la red es lenta), el buffer de envío de A se llena. En un socket **bloqueante**, el `write()` **se bloquea**: la aplicación se frena. Esa es la otra mitad del control de flujo, y es puramente local: no hace falta ningún campo en la cabecera, es el sistema operativo frenando a la aplicación.

El caso del receptor. Simétrico: si B no llama a `read()`, su buffer se llena, el `rwnd` baja, y A se frena. Acá sí hay señalización en la red, porque **el `rwnd` viaja en cada segmento**.

Por qué importa decirlo: los dos casos producen el mismo síntoma observable ("no avanza") pero por mecanismos distintos, y solo uno de los dos necesita el campo `window` de la cabecera.

1.6 Quién controla qué

Variable / mecanismo	Lo decide	Cómo se comunica	Señal explícita
`rwnd`	el **receptor**	campo `window` de cada segmento	sí, el receptor lo anuncia
`cwnd`	el **emisor**	nadie: es una hipótesis local	no, se deduce de ACKs y timeouts
Buffer de envío	el **emisor** (local)	`write()` bloquea	sí, al proceso
Buffer de recepción	el receptor	`read()` consume	sí, al proceso
Segmentación	el **emisor** (MSS del receptor)	opción MSS en el SYN	sí, en el SYN
Cierre ordenado	**ambos**	FIN / TIME-WAIT	sí, FIN

Detalle fino que se pregunta: el `rwnd` lo decide el receptor, **pero es un campo en un segmento que manda el receptor, y lo lee el emisor**. Nadie lo negocia explícitamente: el emisor simplemente obedece lo último que oyó.

⚠️ Actualización a 2026

Sigue igual — de memoria: - `rwnd` viaja en el campo `window` (16 bits) de cada segmento TCP. - `ventana efectiva = min(cwnd, rwnd)`, y es un **mínimo** porque son cotas superiores. - El **zero-window probe** es de **1 byte**, con periodo **exponencial**, y sirve para destrabar, no para retransmitir. - Congestión = problema de **la red**; flujo = problema del **otro host**.

Cambió o se afinó:

- La especificación vigente es **`RFC 9293` (2022, *Internet Standard*)**, que consolida `RFC 793`
- `RFC 879`, `2873`, `6093`, `6429`, `6528`, `6691` y cuyo §8.6 es el de la **persistencia**. Citar `RFC 793` como "la especificación" en 2026 está desactualizado.
- **`Window Scale` (`RFC 7323`, 2014)** es lo que hace que el `rwnd` de 16 bits deje de ser un techo real: se negocia un *shift* en el SYN y el campo queda multiplicado por `2^shift`, hasta `2³°` bytes. Sin esto, con RTT de 80 ms el `rwnd` limitaría el throughput a 6,55 Mbit/s (Clase 19 §1.6).
- **El Initial Window es 10 MSS** por defecto (`RFC 6928`, 2013), no 1. El `rwnd` inicial típico sigue siendo 64 KiB en varias plataformas — que es justamente el valor que **limita antes que el `cwnd`** en conexiones de corta vida. ⇒ **En una descarga corta de web, muchas veces el límite es `rwnd`, no `cwnd`.**
- **El control de congestión por defecto hoy es CUBIC** (`RFC 9438`, 2023, *Standards Track*), no Reno ni Tahoe. Pero el `rwnd` y la fórmula `min(cwnd, rwnd)` **no cambian**: CUBIC decide el valor de `cwnd`, y el mínimo con `rwnd` se sigue aplicando igual. → **Para el examen:** si te piden "el mecanismo de control de congestión", respondé slow start / congestion avoidance / fast retransmit / fast recovery (Clase 21), y si sobra lugar, una línea de "hoy es CUBIC". Si te piden "control de flujo", es `rwnd` y punto.
- **QUIC (`RFC 9000`, 2021)** no usa una única ventana global: maneja **dos niveles**, el de **conexión** (`max_data`) y el de **cada stream** (`max_stream_data`), porque multiplexa muchos streams en una sola

conexión. El *Connection ID* además permite sobrevivir a cambios de topología y al *NAT rebinding*. La idea es la misma; la estructura es otra.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Throughput con el `rwnd` limitado

Datos. `rwnd = 16 KB = 16 384 bytes`, constante. RTT = 120 ms. El `cwnd` es grande (`cwnd = 1 MB`), así que **no** es el que limita.

a) ¿Cuál es la ventana efectiva?

```
ventana efectiva = min(cwnd, rwnd) = min(1 048 576, 16 384) = 16 384 bytes
→ manda el rwnd.
```

b) ¿Cuál es el throughput máximo?

El throughput no es "bytes por RTT": es cuántos bytes salen por segundo. Con 16 KiB cada 120 ms:

```
throughput = 16 384 B / 0,120 s = 136 533 B/s
           = 136 533 · 8 bits/s = 1 092 267 bit/s
           ≈ 1,09 Mbit/s
```

c) ¿Qué `rwnd` necesitaría para alcanzar 10 Mbit/s con ese RTT?

Se despeja de `throughput = rwnd / RTT`:

```
rwnd = throughput · RTT = 10 · 10⁶ bits/s · 0,120 s / 8 = 150 000 bytes
```

Interpretación — y el error conceptual que esconde. Con `cwnd = 1 MB`, **duplicar el `cwnd` no mejora nada**: el mínimo sigue siendo 16 KiB. El único camino es que **el receptor abra el `rwnd`**, es decir que consuma más rápido (`read()` más seguido) o que tenga un buffer de recepción más grande (`SO_RCVBUF`). **Ninguna acción sobre la red sirve para mejorar esto.** Por eso se dice que el `rwnd` es "el grifo que abre el receptor": agrandarlo no le pide nada a nadie.

2.2 Ejercicio 2 — El receptor deja de leer a mitad de la transferencia

Datos. `cwnd = 8 MSS` (8 KiB). El receptor anuncia `rwnd = 16 KiB` al principio. En un momento dado la aplicación de B deja de llamar a `read()` durante 3 segundos.

a) Evolución de la ventana efectiva y del `rwnd`, RTT por RTT.

Suposiciones (y hay que declararlas): MSS = 1 KiB, RTT = 50 ms, sin pérdidas, ACKs inmediatos. "Deja de leer" significa que el buffer de recepción de B no se vacía, pero B sigue mandando ACKs, porque TCP asienta por recepción de bytes, no por `read()`.

```
RTT 0: rwnd = 16 KiB (buffer vacío)     ventana = min(8, 16) = 8 KiB  ← manda cwnd
RTT 1: A manda 8 KiB                     ventana = 8 KiB
RTT 2: B tiene 16 KiB en buffer (8 + 8)  rwnd = 0  → ventana = min(8, 0) = 0
RTT 3..59: ventana = 0                   A NO MANDA NADA
RTT 60: expira el persist timer
        A ──[1 byte, zero-window probe]──►
◄──[ACK=..., rwnd=0]── A                 ventana = 0, espera el doble (100 ms)
... se repite con 200 ms, 400 ms, 800 ms, 1600 ms ...
```

b) ¿Cuántos bytes perdió A en el tiempo en que estuvo frenado?

```
Bytes enviados antes de frenar = 16 KiB (lo que entró en el buffer de B)
Bytes enviados durante el freno = 0
→ A "perdió" ≈ 3 s × 8 KiB/RTT = 3/0,050 × 8 KiB = 480 KiB de oportunidad perdida
```

cálculo de oportunidad, no de datos: esos 480 KiB no se mandaron, simplemente porque la ventana efectiva era 0.

c) ¿Cuándo se reanuda?

```
A manda la sonda → B responde ACK con rwnd > 0
→ ventana efectiva = min(cwnd, rwnd) > 0
→ A reanuda a plena velocidad, sin evento de congestión de por medio
```

d) ¿Qué NO pasó en esta historia?

```
✗ cwnd NO bajó           (no hubo pérdida: no hubo ni un paquete perdido)
✗ No hubo RTO             (no hubo datos sin confirmar: no había nada en vuelo)
✗ La red no se enteró     (no hubo congestión: ningún router bajó nada)
✗ No hubo control de congestión en ningún momento
```

Interpretación. Esta es la demostración más limpia de por qué los dos mecanismos tienen que estar separados. **Todo el throughput perdido se recuperó instantáneamente** en cuanto B volvió a leer, porque la red nunca estuvo congestionada y `cwnd` nunca bajó. Si TCP hubiera aplicado control de congestión, la ventana estaría artificialmente destruida y la recuperación habría tardado varios RTTs.

⚠️ Errores típicos

- **Bajar `cwnd` cuando el `rwnd` es 0.** Es el error conceptual más caro del tema.
- **Asumir que B deja de mandar ACKs cuando deja de leer.** Falso: TCP asienta por **recepción de bytes**, no por entrega a la aplicación. Los ACKs siguen fluyendo (salvo *delayed ACK*), lo que mantiene viva la conexión.
- **Confundir la sonda de 1 byte con una retransmisión.** Es una **consulta**: el byte no es un dato nuevo, es el que ya está confirmado, y su único objetivo es generar un ACK con el `rwnd` actualizado.

2.3 Ejercicio 3 — `cwnd` grande y `rwnd` chico: cuánto throughput se pierde

Datos. Enlace de 1 Gbps, RTT 50 ms. Se comparan tres configuraciones.

a) `cwnd = 1 MiB`, `rwnd = 8 KiB`.

```
ventana efectiva = min(1 048 576, 8 192) = 8 192 bytes
throughput = 8 192 / 0,050 = 163 840 B/s = 1,31 Mbit/s
utilización del enlace = 1,31 / 1000 = 0,13 %
```

b) `cwnd = 8 KiB`, `rwnd = 1 MiB`.

```
ventana efectiva = min(8 192, 1 048 576) = 8 192 bytes
throughput = 163 840 B/s = 1,31 Mbit/s      ← ¡idéntico! y es otro problema
```

c) `cwnd = 1 MiB`, `rwnd = 1 MiB`.

```
ventana efectiva = 1 048 576 bytes
throughput = 1 048 576 / 0,050 = 20 971 520 B/s = 167,8 Mbit/s
utilización = 16,8 %
```

d) Comparación y diagnóstico.

Configuración	Ventana efectiva	Throughput	Utiliz.	**Quién limita**	**Qué se arregla**
`cwnd` =1 MiB, `rwnd` =8 KiB	8 192	1,31 Mbit/s	0,13 %	**el receptor**	que B lea más rápido / `SO_RCVBUF` mayor
`cwnd` =8 KiB, `rwnd` =1 MiB	8 192	1,31 Mbit/s	0,13 %	**el emisor**	que la red dé más (o `cwnd` crezca)
`cwnd` =1 MiB, `rwnd` =1 MiB	1 048 576	167,8 Mbit/s	16,8 %	**el enlace**	más ancho de banda

Interpretación — y acá está la moraleja. Las dos primeras filas dan **exactamente el mismo número**, y sin embargo **ninguna de las dos soluciones sirve para la otra**: si en (a) agrandás `cwnd` a 1 MiB no cambia nada (el mínimo sigue siendo `rwnd`), y si en (b) agrandás `rwnd` a 1 MiB tampoco (ahora el mínimo pasa a ser `cwnd`). Y el 83 % del enlace sin usar en (c) tampoco es culpa de ninguna de las dos ventanas: es el **BDP** (`10⁹ · 0,050 / 8 = 6 250 000 bytes`), así que con 1 MiB de ventana seguís muy por debajo del techo físico.

El corolario práctico: siempre que veas un throughput bajo, **lo primero es mirar los dos límites y ver cuál manda**, antes de tocar nada. Es el mismo diagnóstico del ejercicio 2.2.

3.1 Examen 2009-07, Pregunta 1 (10 puntos)

> **Fuente:** Examen `sirc0907` — 29 de julio de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf **Texto del examen:**
>
> En un protocolo de capa de transporte: a) ¿Cuál es la diferencia entre Control de Flujo y Control de Congestión? b) ¿Cómo hace TCP para detectar la existencia de congestión en la red? c) ¿La pérdida de paquetes siempre implica congestión? Justifique su respuesta.

Respuestas oficiales (textuales):

> a) "El control de flujo intenta que el emisor no desborde el buffer del receptor transmitiendo muy rápido demasiados datos. El control de congestión intenta evitar que se pierdan paquetes debido a que hay demasiadas fuentes enviando muy rápido demasiado tráfico a ser manejado por la red."
>
> b) "TCP asume que la pérdida de paquetes (o grandes retardos) implican congestión."
>
> c) "No. La pérdida de paquetes también puede darse por una capa de enlace defectuosa, por ejemplo."

Parte a) — La diferencia (≈4 puntos)

Se responde con la tabla de §1.1. **El punto clave para el profesor:** los dos se resuelven reduciendo la ventana efectiva, pero por **motivos distintos**, y confundirlos es el error típico. Sumá que el flujo **lo controla el receptor** (y por eso puede anunciarse sin ambigüedad) mientras que la congestión **la tiene que estimar el emisor** (y por eso es una hipótesis, no una medición).

Parte b) — Cómo detecta TCP la congestión (≈3 puntos)

Las dos señales (desarrolladas en la Clase 21):

```
┌──────────────────────────────────────────────────────────┐
│ 1. Timeout (vence el RTO)                                │
│    → no llegó ni el ACK siguiente → pérdida GRAVE o ruta caída│
│                                                          │
│ 2. 3 ACKs duplicados                                     │
│    → el siguiente sí llegó → pérdida AISLADA, la red aguanta  │
└──────────────────────────────────────────────────────────┘
```

La respuesta oficial además menciona **"(o grandes retardos)"**, y eso vale: un RTT que sube mucho indica **cola** en los routers. Pero ojo, el enunciado es explícito: "*TCP asume que...*". Es una **inferencia**, no una notificación. **Con ECN hay una tercera vía**: el router marca CE y el receptor avisa con CONG_DETECTED, que ya no es inferencia sino notificación explícita (Clase 21 §1.6).

Parte c) — ¿Siempre la pérdida implica congestión? (≈3 puntos)

No. Fuentes de pérdida que **no** son congestión:

- **Errores de CRC** en un enlace (ruido, cable malo): el paquete ni siquiera llegó a la cola.
- **Buffer del receptor lleno** (control de flujo): el paquete se descartó aguas abajo, pero la red estaba perfecta.
- **Descarte por ACL / firewall / filtrado** de un router intermedio.
- **Fragmentación descartada** en IPv4 en un router que no tiene espacio para un fragmento.
- **Corte de un enlace** (un cable, no la carga).

La justificación que da puntos: aunque la pérdida *no* siempre sea congestión, **TCP asume que lo es** porque es la apuesta con mejor promedio. Ante la duda entre "la red está saturada" y "fue un error

puntual", **bajar la ventana** cuesta rendimiento en el caso "error puntual" pero **salva la red** en el caso "saturada". El error hacia el otro lado (mantener la ventana y saturar la red) es mucho más caro y además causa *bufferbloat* y colapso. La asimetría de costos justifica la apuesta conservadora.

> **Tope para 10 puntos:** la definición de cada uno, la tabla de comparación, las dos señales de detección, y el "no" de (c) con al menos dos fuentes de pérdida no congestiva y la justificación de por qué TCP asume.

3.2 Examen 2010-02, Pregunta 5 (7 puntos) — el mecanismo de control de flujo

> **Fuente:** Examen `sirc1002` — 2 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf

⚠️ **Nota de método, leela antes de estudiar esto:** el `BANCO-EXAMENES.md` registra `2010-02 P5` como *"jitter vs. retardo"*, y el texto extraído del PDF confirma que ése es el tema. **El enunciado de ese apartado NO es sobre control de flujo.** Lo que sigue es una **solución propia** *(solución propia)* del mecanismo de control de flujo de TCP, desarrollada porque es un tema del bloque y porque aparece explícitamente en `2009-07 P1a`.

Respuesta (7 puntos): el control de flujo tiene **un solo campo**, el `window` de 16 bits en la cabecera, que rellena el **receptor** en cada segmento que manda.

- **a) Quién lo controla y cómo se comunica (2 pts).** El **receptor** es el único que sabe cuánto le queda en su buffer. Como no hay forma de que el emisor lo adivine, lo **anuncia** en el campo `window` de cada segmento TCP que envía (ACKs incluidos), y el emisor **lee** ese campo y se autoimpone el límite.
- **b) La fórmula y el sentido (2 pts).** `rwnd = espacio libre en el buffer + bytes recibidos sin entregar`, y `ventana efectiva = min(cwnd, rwnd)`. Es un **mínimo** porque ambas son cotas superiores independientes: sumarlas desbordaría el buffer del receptor justo cuando la red va peor.
- **c) El caso `rwnd = 0` (2 pts).** Si el receptor no lee, su buffer se llena y el `rwnd` llega a 0. TCP entra en **persistencia** (`RFC 9293` §8.6): el emisor manda **zero-window probes** de **1 byte**, con periodo **exponencial** (empieza en un RTO y duplica hasta un tope), hasta que el receptor responda con un `rwnd > 0`. **No es control de congestión**: la red no tiene nada saturado y bajar el `cwnd` no haría que la aplicación del receptor lea más rápido.
- **d) El otro extremo (1 pt).** Simétricamente, si el **emisor** escribe más rápido de lo que la conexión puede sacar, su **buffer de envío** se llena y el `write()` se bloquea (o devuelve `EAGAIN` en modo no bloqueante). Ese es el otro lado del control de flujo, puramente local.

3.3 Conexión con 2011-03 P4 (10 puntos)

> **Fuente:** Examen `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

En el diagrama clásico de `2011-03 P4` (resuelto por completo en la **Clase 22**, Ejercicio 3), el `cwnd` es lo que **cambia** entre los dos apartados:

- **Sin pérdida:** el `cwnd` **no baja**, y los 4 segmentos salen en un solo vuelo.
- **Con pérdida del primer segmento:** al detectar la pérdida, el `cwnd` **se reduce** (`cwnd/2` en Reno por 3 dupACKs; `cwnd = 1 MSS` en Tahoe o ante timeout). Eso es control de **congestión**, y es la razón por la que los cuatro segmentos **no** salen juntos.

Lo que hay que dejar claro para el examen: en `2011-03 P4a` los 4 segmentos salen juntos porque la ventana **alcanzaba**; en `2011-03 P4b` el primero se pierde, se detecta, y la ventana **ya no alcanza para mandar todo de una**. Un mismo `rwnd` en los dos casos; lo que cambia es el `cwnd`.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Flujo** = el receptor no puede absorber → lo controla `rwnd`. **Congestión** = la red no puede transportar → lo controla `cwnd`.
2. El `rwnd` viaja en el campo `window` (16 bits) de **cada** segmento que manda el receptor.
3. `rwnd = espacio libre + bytes recibidos sin entregar` → es una **cota superior**, no una medida instantánea.
4. `ventana efectiva = min(cwnd, rwnd)`: ambas son cotas, y el `min` garantiza que **nadie se desborda**.
5. **No es la suma**: con la suma, lo que la red no tomó se acumularía en el buffer del receptor y lo desbordaría **justo cuando la red va peor**.
6. `rwnd = 0` → **persistencia** → **zero-window probes** de **1 byte**, periodo **exponencial**.
7. Eso **NO es control de congestión**: la red está perfecta y bajar `cwnd` no haría leer más rápido al otro.
8. El otro lado: si el **emisor** no lee rápido, su buffer de envío se llena y el `write()` se **bloquea**.
9. `cwnd` grande y `rwnd` chico y `cwnd` chico con `rwnd` grande dan **el mismo throughput** y **arreglos distintos**: siempre hay que ver cuál limita.
10. En 2026: `min(cwnd, rwnd)` no cambia; lo que cambió es *Window Scale* (`RFC 7323`), el IW de 10 MSS (`RFC 6928`) y que el `cwnd` lo maneja CUBIC (`RFC 9438`).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la ventana efectiva es el **mínimo** y no la suma?

<details>
<summary>Respuesta</summary>
</details>

2. El receptor anuncia `rwnd = 0`. ¿Qué hace el emisor?

<details>
<summary>Respuesta</summary>
</details>

3. Una conexión tiene `cwnd = 64 KiB` y el receptor anuncia `rwnd = 4 KiB`. El throughput medido es `4 192 / 0,100 s = 41 920 B/s`. ¿Qué pasa si el `cwnd` sube a 1 MiB?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué TCP asume que una pérdida de paquetes implica congestión, si no siempre es así?

<details>
<summary>Respuesta</summary>
</details>

5. Nombra dos causas de pérdida de paquetes que **no** son congestión.

▶ Respuesta

6. ¿Quién decide el `rwnd`? ¿Cómo se lo comunica al emisor? ¿Y por qué el emisor no puede ignorarlo?

▶ Respuesta

7. El `cwnd` es de 8 KiB y el `rwnd` de 1 MiB. El throughput es 1,31 Mbit/s con RTT 50 ms. ¿Se arregla agrandando el `rwnd`?

▶ Respuesta

8. ¿Qué cambió entre 2011 y 2026 en este tema?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc0907` — P1 (10 pts): flujo vs. congestión, detección de congestión, y si toda pérdida es congestión
- **Examen** `sirc1002` — P5: jitter vs. retardo (el `BANCO-EXAMENES.md` lo registra así; §3.2 desarrolla el control de flujo como solución propia)
- **Examen** `sirc1103` — P4 (10 pts): el diagrama de 4 segmentos y el papel del `cwnd`
- **Examen** `sirc0912` — P2 (10 pts): fiabilidad en transporte vs. enlace
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 §6.7 "TCP Congestion Control" (pág. ~600 del PDF)
- `RFC 9293` — TCP (2022, *Internet Standard*); §8.6 persistencia / *zero-window probes*
- `RFC 7323` — Timestamps y Window Scale (2014, obsolesce `RFC 1323`)
- `RFC 6928` — Initial Window = 10 MSS (2013) · `RFC 5681` — TCP Congestion Control (2009)
- `RFC 9438` — CUBIC (2023, *Standards Track*, obsolesce `RFC 8312`)
- `RFC 9000` — QUIC (2021): control de flujo por conexión y por stream
- **Clase siguiente:** 21 — Control de congestión en TCP: slow start, AIMD, fast recovery y ECN